

1

```
MariaDB [EmployeesDB]> CREATE PROCEDURE departamentos_canada()  
-> BEGIN  
-> SELECT COUNT(*) AS num_departamentos  
-> FROM Departments d  
-> JOIN Locations l ON d.location_id = l.location_id  
-> JOIN Countries c ON l.country_id = c.country_id  
-> WHERE c.country_name = 'Canada';  
-> END //  
Query OK, 0 rows affected (0.023 sec)
```

2.1

```
MariaDB [EmployeesDB]> CREATE TABLE IF NOT EXISTS last_employees (  
-> employee_id INT(11) PRIMARY KEY  
-> );  
Query OK, 0 rows affected, 1 warning (0.001 sec)
```

2.2

```
MariaDB [EmployeesDB]> CREATE PROCEDURE insertar_last_employees()  
-> BEGIN  
-> DELETE FROM last_employees;  
-> INSERT INTO last_employees (employee_id)  
-> SELECT employee_id FROM Employees  
-> ORDER BY employee_id DESC  
-> LIMIT 10;  
-> END //  
Query OK, 0 rows affected (0.018 sec)  
  
MariaDB [EmployeesDB]> DELIMITER ;  
MariaDB [EmployeesDB]>  
MariaDB [EmployeesDB]> CALL insertar_last_employees();  
Query OK, 20 rows affected (0.024 sec)
```

3.

```
MariaDB [EmployeesDB]> CREATE FUNCTION iniciales_empleados()
-> RETURNS VARCHAR(1000)
-> BEGIN
->   DECLARE fin INT DEFAULT 0;
->   DECLARE v_first_name VARCHAR(20);
->   DECLARE v_last_name VARCHAR(25);
->   DECLARE resultado VARCHAR(1000) DEFAULT '';
->
->   DECLARE cur CURSOR FOR SELECT first_name, last_name FROM Employees;
->   DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;
->
->   OPEN cur;
->   WHILE fin = 0 DO
->     FETCH cur INTO v_first_name, v_last_name;
->     IF fin = 0 THEN
->       SET resultado = CONCAT(resultado, LEFT(IFNULL(v_first_name, '?'), 1), LEFT(v_last_name, 1), ' ');
->     END IF;
->   END WHILE;
->   CLOSE cur;
->
->   RETURN resultado;
-> END //
Query OK, 0 rows affected (0.023 sec)

MariaDB [EmployeesDB]> DELIMITER ;
MariaDB [EmployeesDB]>
MariaDB [EmployeesDB]> SELECT iniciales_empleados() AS iniciales;
+-----+
| iniciales |
+-----+
| SK NK LD AH BE DA VP DL NG DF JC IS JU LP DR AK SB ST GH KC MW AF PK SV IM JR KP JT JL KG CJ SB BE JW MH PF SM HB SH WG |
+-----+
1 row in set (0.011 sec)
```

4.

```
MariaDB [EmployeesDB]> CREATE PROCEDURE aumentar_salario_minimo(IN aumento DECIMAL(8,2), OUT total_salarios DECIMAL(10,2))
-> BEGIN
->   UPDATE Jobs
->   SET min_salary = min_salary + aumento
->   WHERE min_salary < 40000;
->
->   SET total_salarios = (SELECT SUM(min_salary) FROM Jobs);
-> END //
Query OK, 0 rows affected (0.034 sec)

MariaDB [EmployeesDB]> DELIMITER ;
MariaDB [EmployeesDB]>
MariaDB [EmployeesDB]> CALL aumentar_salario_minimo(500, @total);
Query OK, 19 rows affected (0.006 sec)

MariaDB [EmployeesDB]> SELECT @total AS suma_salarios_minimos;
+-----+
| suma_salarios_minimos |
+-----+
| 229300.00 |
+-----+
1 row in set (0.001 sec)
```

5.Tabla error

```
MariaDB [EmployeesDB]> CREATE TABLE errores_salario (
->   id INT(11) AUTO_INCREMENT PRIMARY KEY,
->   empleado VARCHAR(50),
->   mensaje VARCHAR(200),
->   fecha DATETIME
-> );
Query OK, 0 rows affected (0.030 sec)
```

5. Trigger

```
MariaDB [EmployeesDB]> DELIMITER //
MariaDB [EmployeesDB]> DROP TRIGGER IF EXISTS antes_aumento_salario //
Query OK, 0 rows affected, 1 warning (0.001 sec)

MariaDB [EmployeesDB]> CREATE TRIGGER antes_aumento_salario
-> BEFORE UPDATE ON Employees
-> FOR EACH ROW
-> BEGIN
->     DECLARE v_min_salary DECIMAL(8,2);
->     DECLARE v_max_salary DECIMAL(8,2);
->
->     SELECT min_salary, max_salary INTO v_min_salary, v_max_salary
->     FROM Jobs
->     WHERE job_id = NEW.job_id;
->
->     IF NEW.salary < v_min_salary OR NEW.salary > v_max_salary THEN
->         INSERT INTO errores_salario (empleado, mensaje, fecha)
->         VALUES (CONCAT(NEW.first_name, ' ', NEW.last_name), 'El salario esta fuera del rango para este puesto', NOW());
->     END IF;
-> END //
```

Query OK, 0 rows affected (0.058 sec)